AML Final Report

Sight&Sound

Sahil Shah (160050005)
Preey Shah (160050008)
Naman Jain (160050025)
Aman Bansal (160050028)

1 Project Goals

The main goal of the project is to generate a relevant sound from an image and vice versa i.e. mapping image to sound. We do so by mapping both sound and image to a common latent space. For example, if we have an image of a dog, we should ideally be able to generate a sound of a dog barking. For this, we have a dataset of image and audio files, and we train them so that those having the same objects ("classes") go to a shared latent space and then generate the sound or image (as required). The most important feature of our approach was that we wanted to use **non-autoregressive models** in our predictions. For sound generation, most of the popular approaches (such as Wavenet, SampleRNN) are all autoregressive, assuming full dependencies and therefore very slow. On the contrary we wanted to explore more of the recent works that use one-shot GANs for generation sound generation.

Through this project, we got an idea of how machine learning is applied to sound. We also understood on how to work in case there is no direct dataset but we have different datasets in which both can be projected to the same latent space. We also got an insight into generative models like VAE.

2 Related Work

The closest to our work on visual to audio generation comes from [1] who produce audio relevant to a video while we try to do the same for an image. Moreover they used SampleRNN (an autoregressive model) as a part of their audio generation module while we used one-shot generative models. We also use the dataset used in [1] (discussed in 3), by separate out the the audio and the video files and train them separately.

For sound generation we studied the recent state of the art **GANSynth** [2] to understand how GANs are used on sound. Since sound is represented by very large vector, a common approach is to use spectrograms (by taking STFT on the audio vector). **Spectrograms** are represented as a 2 channel image, one

corresponding to magnitude and other for phase. For understanding details of preprocessing sound, constructing spectrograms etc., we used standard online sources. It is a well known fact that learning phase can be very difficult as the phase wraps around. The authors of [2] introduced another representation called Instantaneous Frequency(IF) which they show performs well and are able to generate natural audio examples.
We also referred to [3] for generation of sound through VAE and also to understand the various difficulties in it. For comparisons between models on different types of sound, we have also looked at [4].
For the image part of network we used pretrained Resnet architecture of Imagenet.

3 Dataset

We have used **VEGAS** (*Visually Engaged and Grounded AudioSet*) [1] as our dataset. It includes some selected videos from AudioSet [5] which have good relation between the video and the audio. There are 10 classes in total (Baby crying, Human snoring, Dog, Water flowing, Fireworks, Rail transport, Printers, Drum, Helicopter and Chainsaw) and around 1500-2000 videos for each class with each video having a duration of 10s. Handling audio of length 10s would have been quite difficult because we would have to deal with large-sized spectrograms. To avoid this we divided each 10s audio into 5 audios of 2s each. For each audio we had a 2s video so we chose its midpoint as the image we would be associating with the audio.

The dataset can be found at `http://bvision11.cs.unc.edu/bigpen/yipin/visual2sound_webpage/visual2sound.html`

4 Different approaches

4.1 Pre-processing

It is necessary to convert a wav file into a form that can be used for training. We transform it to a spectrogram using standard libraries and take the log magnitude to make it more sensitive to human hearing. For post-processing, we once again convert the spectrogram back to a wav file using standard libraries and open source code by authors of GANSynth.

4.2 Audio Training

It was possible to use the WaveNet Model as our audio generator. However, it is autoregressive and therefore very slow to train and infer. We were keen to try out non-autoregressive method as we hoped it would lead to faster inference and would train well even on relatively limited resources.

The overall approach involved here was to create a model wherein the two datasets (image and sound) can be trained to map to a common latent space

to allow inter-conversion between the two. Then the overall loss would be sum of the errors due to difference between the latent spaces (KL divergence loss) and the error in the generated output as compared to the original sound (reconstruction error).

We use an architecture inspired by GANSynth to create a decoder that will generate spectrograms from a latent space that is shared between sound and images. We used purely convolutional building blocks that were used in the GANSynth model to build our audio encoders and decoders. We experimented with different loss functions and model architectures.

To achieve this goal, we develop the model incrementally. We first try to train a encoder-decoder model on the audio itself by assuming a standard Gaussian prior on the latent space. Only if this gives good results, do we try to augment it for the image side.

4.2.1 VAE

In VAE, we try to train the sound encoder and decoder by projecting it to a multi-variate Gaussian. We use the standard lower bound error in VAE, assuming a prior of standard multi-variate Gaussian on the latent space. To train the encoder, we use alternating convolution layers and the average pooling layers. The average pooling layer downsamples by a factor of 2 to give the reduced space. At the end, there is a fully connected layer that outputs mean and standard deviations for 128 dimensions. Similarly, the decoder has alternating convolution layers and upsampling (deconvolution) layer.

However, we were not able to reduce the error to a satisfactory level. The error kept blowing up suddenly after training for many iterations for relatively high learning rates. On reducing the learning rate, the model was not able to train well. Therefore, the error does not reduce enough to make the sound distinguishable between the different classes. Note that to test the semantic quality of the sound, we could not figure a method except manually listening to the sound. The quality of sound was satisfactory only for very low values of MSE (under 0.05). There were also various other hyper-parameters to tune :-

- Relative weights of the latent space error and the reconstruction error - This significantly affects the training loss and we adjusted to reduce the loss as much as possible.
- The linearity coefficient in Leaky ReLU
- Parameters like batchsize and **learning rate**

On training the magnitudes and the phase separately, we observed that the model is not able to learn the phase part while the error in magnitude reduces. This was consistent with previous observations about the phase being difficult to train. We also explored directly passing the input phase to the output and only learning the magnitude. This method would not allow generation during inference - and one could rely on methods like performing a nearest neighbour

search on the log magnitudes for all audio samples of the same class in the training set, but it is doubtful that this would provide satisfactory results.

Another issue we had is that the spectrograms that were generated from preprocessing the sound were quite big (192×512). Also, since the dataset was quite large, we did not have access to a GPU that could train a model of sufficient capacity. So , to train it, we reduce the spectrogram to size 48×128, train it, and then upsample it. This method also reduces the accuracy of the model.

We also experimented by reducing the spectrogram to an even smaller size and tried to train the model on this size. The error does come down significantly for such sizes but the audio files generated were very unclear due to the upsampling.

4.3 Bringing in Images

Initially, we planned to train a VAE and then use a pretrained ImageNet network in order to map the images to the same latent space and then apply a contrastive loss based on class to enforce similarity in the latent space of images and sounds from the same class. However, since the VAE model proved difficult to train and gave worse results than a simple autoencoder, we added a **ResNet** pretrained on ImageNet data, replaced its last fully connected layer with a feed-forward network whose output was the same size as our latent space, and then imposed an MSE loss to enforce the latent space representations of corresponding images and sound spectrograms to be as close as possible. Additionally, we added (MSE) losses to enforce both - the spectrograms reconstructed from the image features and those reconstructed after encoding the ground truth sound spectrograms to the latent space - to be close to the ground truth.
This resulted in 3 losses which were given different weights and the network was trained using all losses simultaneously. Finally, we also experimented with using the sound spectrogram encoder and decoder pretrained using only sound autoencoding, but this did not provide a significant advantage. The model without pretraining achieved a similar loss within a few epochs.

4.4 Sound to image

We also tried exploring generation of an image from an audio file. We implemented an audio classifier to train the sound which was performing quite well in terms of accuracy. Then we implemented a GAN-based architecture and fine-tuned the features used by the classifier for generating image samples. Although the features were very good, the generated image were not up to the mark. One reason for this is that images are much more perceptible than audio and differences are clearly visible in images, due to which we require quite high accuracy for decent results, which we were not able to achieve.

5 Experiments

5.1 Code and Environment

We used python language for conducting all our experiments along with necessary shell scripting. We used PyTorch Library (version = 0.4.1) as our Deep Learning Toolkit. We **did not** use any external code apart from external libraries for generating spectrograms for audio and vice-versa. There was about 1200-1400 lines of code conservatively written for writing all models and training scripts.

The code can be browsed here : `https://github.com/Naman-ntc/AudioVision`

5.2 Experimental platform and Hardware

We used a machine on server of Prof. Arjun Jain. We had access to a GPU which was shared among other users as well.

5.3 Experimental Results

5.3.1 Audio

Regarding the audio reconstruction, we were able to get the training error down to 0.05 . Since we do not have any appropriate metric for how good a sound sounds, we had to rely on manual inspection of the generated audio. We saved the reconstructed audio and on manual inspection, we had to interpolate that we needed the error to fall below 0.02 to reconstruct a sound which can be identified to belong to a class. However we tried various approaches and though we got some decent results where we were able to identify the reconstructed audio and the ground truth yet none of the approaches took the training error below the mark.

5.3.2 Image to Sound

We tried various approaches as already mentioned. The sound which was generated using the images was manually inspected but it was not possible to identify the sounds as belonging to a particular class. Though the sounds did have some pattern but they were not expressive enough to be able to get distinguished based on the class.

6 Efforts

6.1 Fraction of Time for Different Parts

Since sound was a new domain for all the members it took large chunks of time initially to start. A substantial part of the time initially was taken up in

understanding how to process sound, input representation, constructing spectrograms that enable noiseless reconstruction when converted back to audio. It also took us time to understand the standard GANSynth related in processing sound. This was new for us so understanding this took time. There were some standard libraries available for converting wav file. However, significant time was taken up in installing and running these libraries.
As most members of the team were in general aware of image processing and hence that did not take up much time.
Writing the training procedure and code for the model took up a significant time. Most of our time was taken up in experimenting with the model, figuring out why it is not working, and implementing different ideas. Moreover due to large size of spectrograms it took a lot of time to train (about a half day for each experiment) and we performed many experiments listed below

- L1 loss, L2 loss, VAE loss
- Normalization techniques (with all loss techniques)
- Phase & spectrogram different normalization
- Train only phase or only magnitude
- Changing size of spectrograms
- Pass phase directly
- Image to sound
- Sound to image

These experiments along with changing various hyper-parameters got us investing a lot of time in the project.

6.2 Most Challenging Part

The most challenging part was training the **phase** of the audio, in accordance with observations in similar previous experiments. Most of our efforts were directed towards reducing this error. Another issue was the capacity of our models, as the size of the spectrogram was quite large.

6.3 Fraction of Work Done by Different Members

There was no specific work division. Every member did whatever work was there at hand. All the members equally contributed to the project.

References

[1] Yipin Zhou, Zhaowen Wang, Chen Fang, Trung Bui, and Tamara L. Berg. Visual to sound: Generating natural sound for videos in the wild. *CoRR*, 2017.

[2] Jesse Engel, Kumar Krishna Agrawal, Shuo Chen, Ishaan Gulrajani, Chris Donahue, and Adam Roberts. Gansynth: Adversarial neural audio synthesis. *CoRR*, abs/1902.08710, 2019.

[3] Yu-An Chung, Wei-Ning Hsu, Hao Tang, and James Glass. An unsupervised autoregressive model for speech representation learning. *CoRR*, abs/1904.03240, 2019.

[4] Fanny Roche, Thomas Hueber, Samuel Limier, and Laurent Girin. Autoencoders for music sound synthesis: a comparison of linear, shallow, deep and variational models. *CoRR*, abs/1806.04096, 2018.

[5] Jort F. Gemmeke, Daniel P. W. Ellis, Dylan Freedman, Aren Jansen, Wade Lawrence, R. Channing Moore, Manoj Plakal, and Marvin Ritter. Audio set: An ontology and human-labeled dataset for audio events. In *2017 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP 2017, New Orleans, LA, USA, March 5-9, 2017*, pages 776–780. IEEE, 2017.